Why Math for AI

Three mysteries, one destination, and the course map

Prof. Nipun Batra

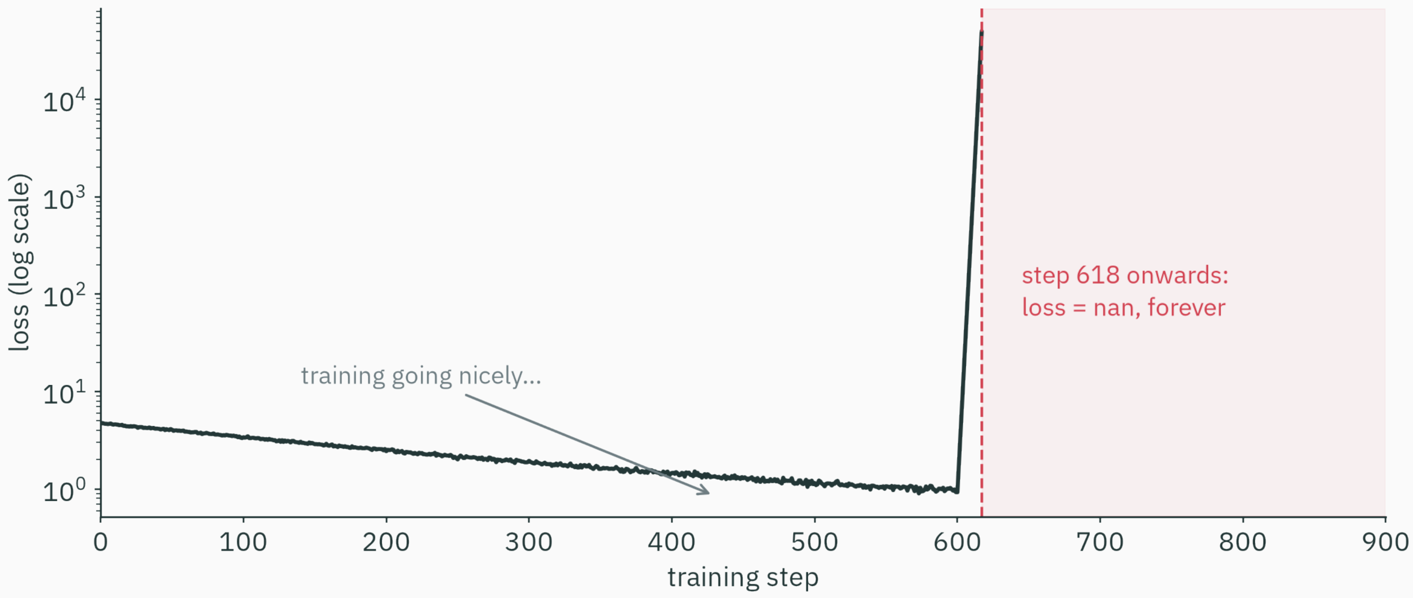
Mathematical Foundations for AI · IIT Gandhinagar

The hook: three mysteries

Mystery 1 · the loss that became NaN

V VISUAL

You train a model. The loss falls beautifully for 617 steps. Then —



No error message. No crash. Just `nan`, forever. **What happened?**

Mystery 1 · a three-line crime scene

The model's last layer computes a **softmax** — it turns scores into probabilities:

```
>>> import numpy as np
>>> z = np.array([1000., 1001., 1002.])   # perfectly reasonable scores
>>> np.exp(z) / np.exp(z).sum()
array([nan, nan, nan])
```

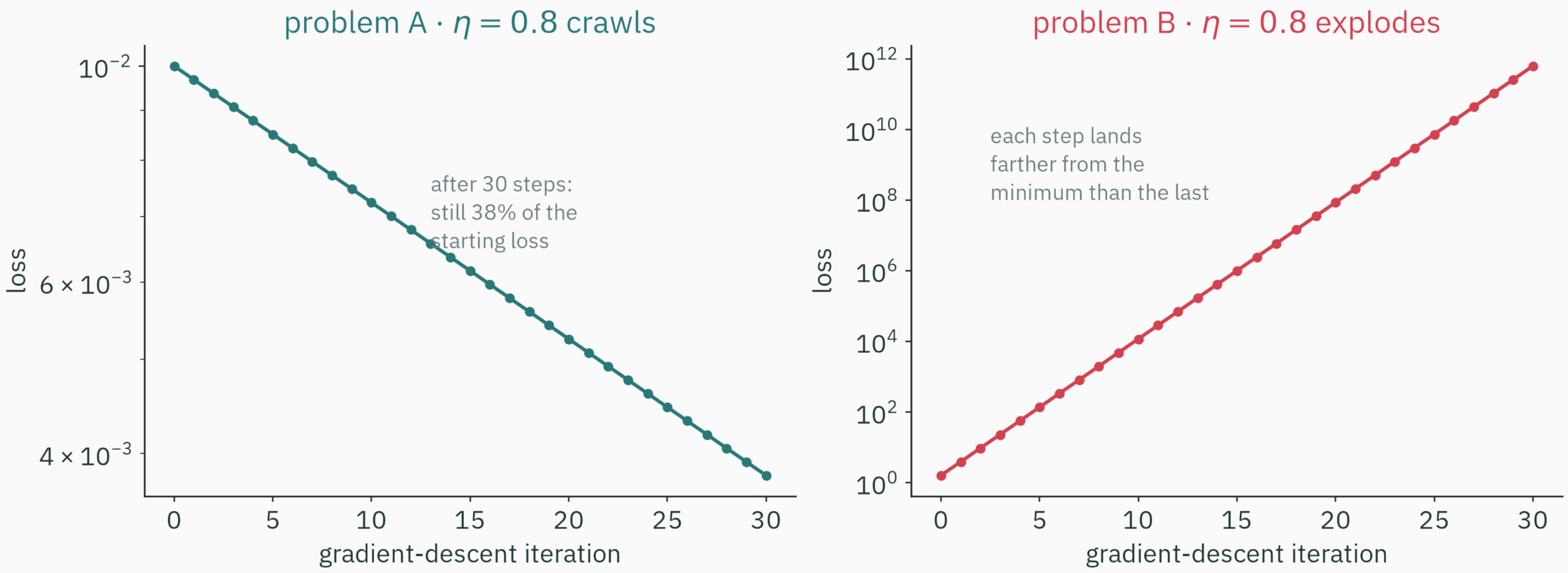
e^{1000} is larger than **any number this machine can store** — so it gives up.

> Real numbers and machine numbers are **not the same thing**. The gap between them is **Lecture 2** — the very next class.

Mystery 2 · one learning rate, two personalities

V VISUAL

Gradient descent is the workhorse of all machine learning. Same code, same learning rate $\eta = 0.8$ — on two different problems:

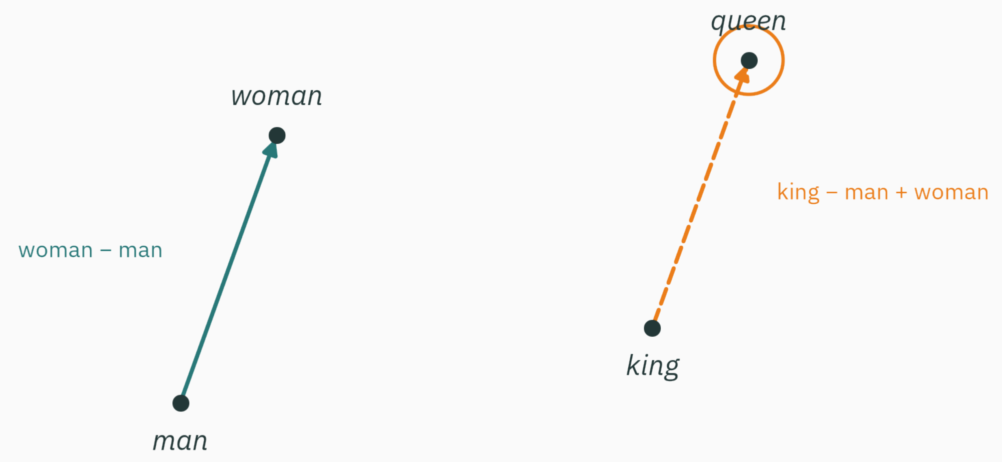


It crawls on one and explodes on the other. **Why?** Resolved in **L17** (conditioning).

V VISUAL

Modern AI stores every word as a list of ~300 numbers (an **embedding**). And then this works:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$



a 2-D shadow of a 300-dimensional embedding space (→ L3)

Mystery 3 · arithmetic on meaning

V VISUAL

How can **subtracting lists of numbers** capture *gender*? Resolved in **L3** (vectors).

Three mysteries, three lectures

Mystery	The math that explains it	Where
the loss goes `nan`	floating-point numbers	**L2** — next class!
the same η crawls *and* explodes	conditioning, curvature	**L17**
king − man + woman ≈ queen	vector geometry	**L3**

Every lecture opens with a real AI failure or capability — and closes by explaining it. The math is never decoration; it is the explanation.

The AI stack

Every AI system is the same five boxes

V VISUAL

what breaks without the math

“features on wrong scales”	“singular matrix”	“loss = nan”	“divergence”	“float16 overflow”
DATA	**MODEL**	**LOSS**	**OPTIMIZER**	**DEPLOYMENT**
vectors · matrices	linear maps · distributions	likelihood · cross-entropy	gradients · convexity	floating point · compression
M1 · L3–L6	*M1 & M3*	*L14 · L22–L24*	*M2 & M4*	*L2 · L23*

the math each stage runs on — and the lectures that deliver it

The stack, stage by stage

Stage	The math it runs on	Where we build it
Data	vectors, matrices, norms	L3–L6 (stored as floats: L2)
Model	linear maps, probability distributions	L4–L6, L12–L13, L25
Loss	likelihood, cross-entropy	L14, L22–L24
Optimizer	gradients, autodiff, convexity	L7–L11, L16–L21
Deployment	floating point, compression	L2, L23

This table is the course: five stages, six modules, one destination.

The destination

By Lecture 26, you will train this

A **character-level language model**, from scratch, on the complete works of Shakespeare. Its actual output:

```
DUKE VINCENTIO:
Well, your wit is in the care of side and that.

CLARENCE:
And so the sight the world and the more,
I shall be my lord the king of hearth.
```

Not GPT — but the **same mathematical species**: a probability distribution over text, fit by maximum likelihood, scored by cross-entropy, trained with gradients, computed in floating point.

Every module is a load-bearing wall

What that little language model needs from each module:

Module	**What the language model needs from it**
0 · Machine numbers	probabilities underflow → log-space, log-sum-exp
1 · Linear algebra	transition matrices, stationary distributions
2 · Calculus + autodiff	gradients of the loss, backprop
3 · Probability + MLE	the model *is* a distribution; fitting = MLE
4 · Optimization	gradient descent actually finds the parameters
5 · Information theory	cross-entropy loss, perplexity
6 · Markov chains	the model class itself

We check the destination at every module boundary

After module...	...you can build this much of it
0 · machine numbers	store its probabilities without underflow
1 · linear algebra	run its transition matrix, find steady states
2 · calculus + autodiff	differentiate its loss automatically
3 · probability + MLE	define its loss (and know where losses come from)
4 · optimization	actually train it
5 · information theory	evaluate it (perplexity, compression)
6 · Markov chains	assemble the whole thing — and generate Shakespeare

A course with a visible destination feels designed, not assembled.

The teaching contract

How every idea in this course will arrive

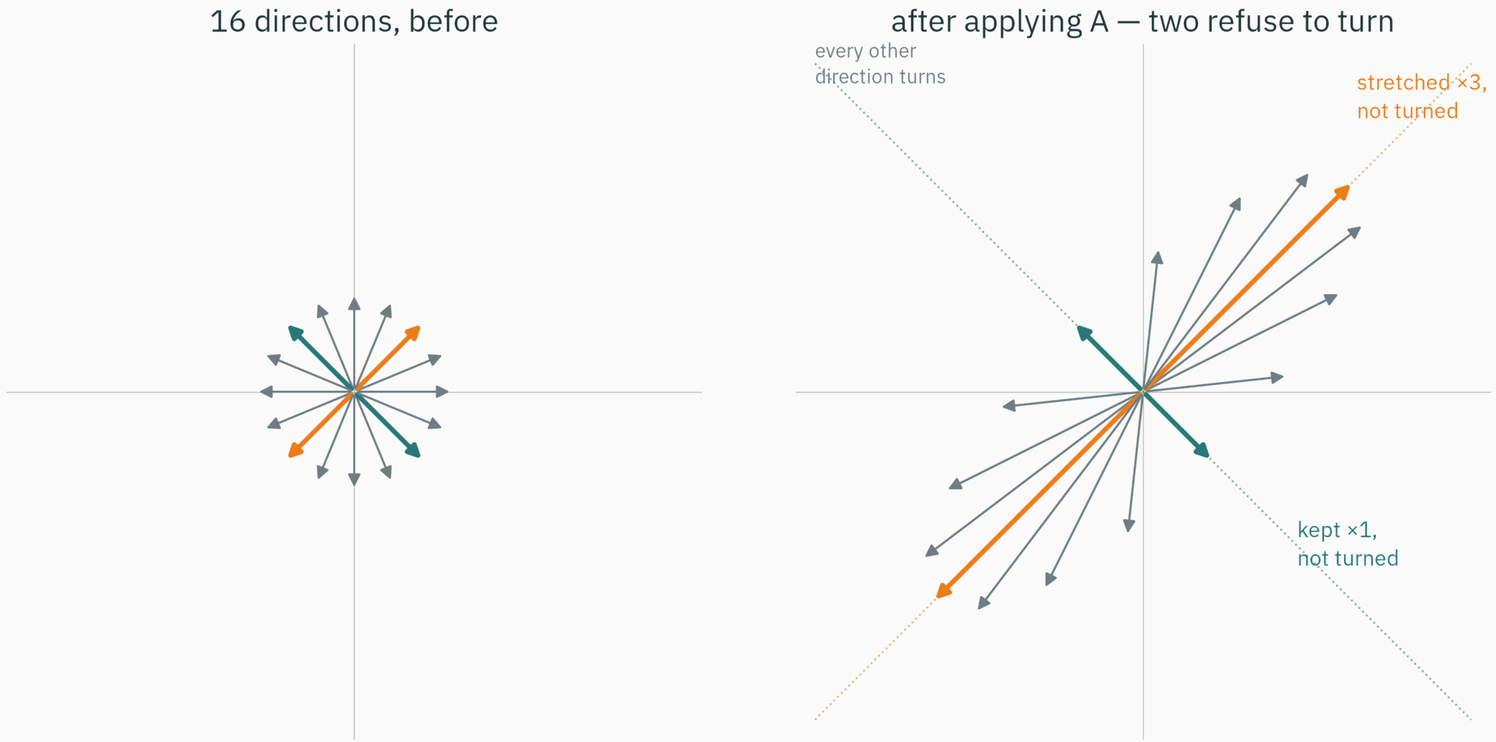
1. **Picture** — a geometric intuition you can *see*
2. **Numbers** — a worked example small enough to check by hand
3. **Code** — the same example in ~5 lines of NumPy
4. **Symbols** — only *now*, the general definition

> If you ever meet a formula in this course and think “where did that come from?” — that is a bug in the course. Report it.

Let’s demo the contract right now, on a concept from **L5**: *eigenvectors*.

Step 1 · picture: the directions that don't turn

A matrix moves every vector. Watch 16 directions, before and after:



Almost every direction gets **turned**. Two directions refuse.

Step 2 · numbers: check it by hand

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

The special direction $\boldsymbol{v} = (1, 1)$:

$$A\boldsymbol{v} = \begin{pmatrix} 2 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 1 + 2 \cdot 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix} = 3\boldsymbol{v} \quad \text{same line, stretched } \times 3$$

An ordinary direction $\boldsymbol{u} = (1, 0)$:

$$A\boldsymbol{u} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad \text{knocked off its line — turned}$$

Step 3 · code: three lines of NumPy

```
>>> A = np.array([[2., 1.], [1., 2.]])
>>> vals, vecs = np.linalg.eig(A)
>>> vals
array([3., 1.])
>>> vecs[:, 0]                    # the (1,1) direction, normalized
array([0.70710678, 0.70710678])
```

The machine found both special directions — and how much each one stretches.

Step 4 · symbols: only now, the definition

A nonzero vector v is an **eigenvector** of A, with **eigenvalue** λ, if

$$Av = \lambda v$$

— applying A only *scales* v; it never turns it.

You just previewed L5 — and seen how every concept will arrive.
Picture first. Symbols last. Always.

The fine print

Prerequisites · an honest accounting

We assume (ES 113 + ES 114)

- Python + NumPy — loops, arrays, plotting
- discrete probability — coins, dice, Bayes on events
- descriptive statistics — mean, variance
- school calculus — $\frac{\mathrm{d}}{\mathrm{d}x}x^n$, one variable

We do NOT assume

- any machine learning
- linear algebra beyond 12th grade
- multivariable calculus
- continuous distributions / densities
- optimization of any kind

> If you can read a `for` loop and reason about a coin flip, you have everything you need on day one.

How the course runs

What	How much	Notes
Lectures	26 × 80 min	intuition first, an AI hook every time
Tutorials	13 × 80 min	**hybrid**: ~40 min worksheet + ~40 min notebook
Credit	3–1–0–4	tutorials are formative — effort counts, not marks
Assessment	quizzes + midsem + endsem	a quiz after (nearly) every module

Worksheets build pen-and-paper fluency (exam style); notebooks rebuild the *same idea* computationally. You need both hands.

The textbooks · all excellent, all free

Book	**Role in this course**
Deisenroth, Faisal, Ong — *Mathematics for ML*	the backbone (Modules 1–4)
Boyd & Vandenberghe — *Convex Optimization*	Module 4 (optimization)
MacKay — *Information Theory, Inference & Learning*	Module 5 (information theory)
Solomon — *Numerical Algorithms*	Module 0 + numerics throughout

All four are **legally free PDFs** from the authors: mml-book.github.io · stanford.edu/~boyd/cvxbook · inference.org.uk/itila · Solomon's *Numerical Algorithms*. Reading pointers close every lecture.

The course site · everything in one place

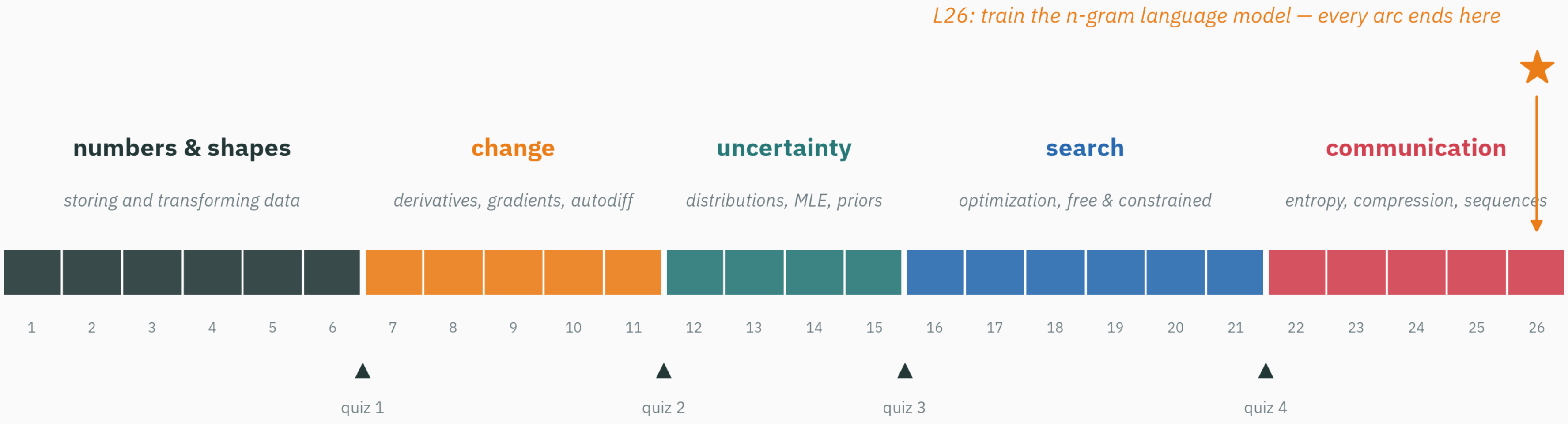
- **Slides** — PDF for every lecture, posted before class
- **Worksheets** — pen-and-paper problem sets, with solutions
- **Notebooks** — every tutorial's computational half, Colab-ready
- **Interactives** — browser toys for the big ideas (eigenvectors, gradient descent, IEEE-754, …)

Tutorial 1 lands right after L2: IEEE-754 by hand (worksheet) + floating-point experiments in NumPy (notebook). Tutorials always follow the lecture that feeds them.

The map

The course at a glance

VISUAL



The narrative arc

Lectures	**Arc**	**The question it answers**
L1–L6	**numbers & shapes**	how do machines store data — and transform it?
L7–L11	**change**	how do we measure sensitivity — and automate it?
L12–L15	**uncertainty**	how do we model data with distributions — and fit them?
L16–L21	**search**	how do we find the best parameters, free and constrained?
L22–L26	**communication**	entropy, compression, sequences — and the finale

The narrative arc

The keystone arrives in L14: *every loss function is a negative log-likelihood in disguise*.

Checkpoint: calibrate yourself

Q QUESTION

No marks — just checking that the prerequisites are alive:

Three warm-ups. Commit to answers for all three before the next slide.

A. You flip a fair coin twice. P(two heads)?

B. What does `np.dot([1, 2], [3, 4])` return?

C. What is the slope of $f(x) = x^2$ at $x = 3$?

D. (no fourth question — breathe)

Answers: calibrate yourself

A ANSWER

A. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$ — independent events multiply (ES 114). By L12 you'll do the *continuous* version.

B. $1 \cdot 3 + 2 \cdot 4 = 11$ — the dot product. In L3 it becomes *geometry*: length, angle, similarity.

C. $f'(x) = 2x$, so 6 — school calculus. In L7 the derivative becomes *the best local linear model*.

> If all three felt easy, you are fully prepared. If one wobbled, skim your ES 113/114 notes this week — these are the only prerequisites we lean on.

Lecture 1 — summary

- **AI fails (and works) in mathematically explainable ways** — NaN losses (L2), diverging optimizers (L17), embedding arithmetic (L3).
- **The stack**: data → model → loss → optimizer → deployment — each stage runs on specific math.
- **The destination**: a character-level language model, trained from scratch in L26.
- **The contract**: picture → numbers → code → symbols — demoed today on eigenvectors.
- **Prerequisites**: ES 113/114 only; no ML assumed. Four textbooks, all free.

Read before L2 — nothing required. If curious: Goldberg, *What Every Computer Scientist Should Know About Floating-Point Arithmetic* (1991), §1.

Every AI system is a stack of math — by L26 you'll have built one from scratch.

Next: **floating point** — Mystery 1 gets solved: how machines actually store numbers, and why e^{1000} broke ours.